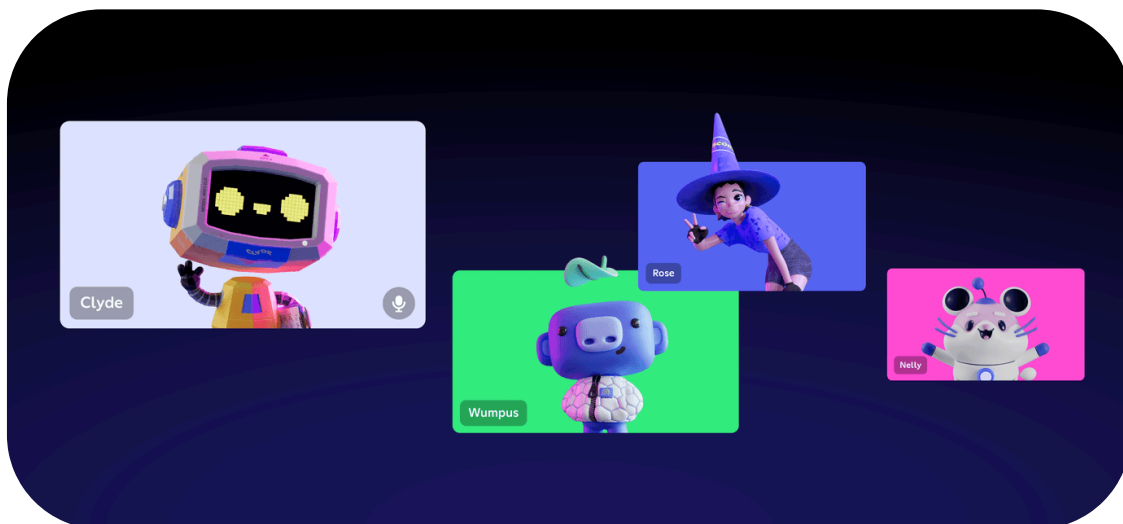




Engineering & Developers

How We Moved Discord Voice to the Edge



David Chen

June 9, 2026

At Discord, the distance to a user's closest voice server matters. Every millisecond of network distance adds latency to every packet, and past a certain point calls stop feeling like your friend is in the same room as you.

For most of Discord's history, the closest voice server we could put you on was in one of about 30 cities worldwide, in places where the major cloud providers had data centers. That worked fine if you lived in the Bay Area or Frankfurt, and less well if you lived in Reykjavik, Auckland, or other places where hyperscaler coverage was thin.



Discord

Open Discord

Last year, we started migrating Discord voice and video traffic onto Cloudflare's edge network, which runs in over 300 cities. Today, more than 80% of our voice and video traffic runs there, and 70% of our regions show year-over-year quality improvements. Frankfurt leads the way, with ping averages down 34% and packet loss down 42% compared to the previous vendor.

This post is the story of how we got here: why we did it, what we had to build to make it work, and how we investigated quality issues in Europe earlier this year.

A Different Shape of Network

Most cloud infrastructure is built around a small number of large data centers. Cloudflare went the other way: smaller points of presence (PoPs) in over 300 cities, and their pitch was to run Discord's voice and video software in all of them.

CDNs are built for caching static content close to users, but Discord's voice and video services route live, low-latency UDP packets between everyone in a call in real time. Cloudflare grew out of CDN architecture and had built enough of their platform out for a wide variety of traffic that we thought it could work for our workflows too.

The change opens up the geography for us. Traditionally, an Icelandic user's call routes through Rotterdam, several hundred kilometers away. Cloudflare runs a PoP in Reykjavik. Every new location is still a deployment on our side, but for the first time, geography isn't the limiting factor. Same for New Zealand, Hawaii, Lagos, and a lot of other places that used to route through whichever hyperscaler region was closest. For comparison, other hyperscale cloud providers operate approximately 30 to 40 regions globally, leaving certain countries uncovered, while Cloudflare's network spans 300+ PoPs across all major cities.

Two Regions, Two Surprises

Before starting any engineering work, we had to test whether the basic geography premise held up. We expected to drop Cloudflare PoPs in to replace the prior provider's regions and let traffic naturally

shift over. The first two rollouts taught us the premise behind it didn't hold.



[Open Discord](#)

Phase 0: Iceland

In late February 2025, Iceland was our first test. Our prior voice and video provider didn't have a presence there, so Icelandic users had been routing through Rotterdam, which is several hundred kilometers away. Cloudflare has a PoP in Reykjavik, so we set up some Voice Servers there and watched what happened.

For Iceland-only calls, the numbers improved as expected. Ping dropped 9%, packet loss dropped 11%. Local users were hitting a local server.

But mixed-region calls, the numbers went the other way. Ping for non-Iceland users on Iceland-hosted calls jumped 2.7x. Packet loss climbed 9%.

This was happening due to how Discord is designed to assign calls to servers. We pick one SFU instance to host an entire call, and every participant connects to that one host for the call's entire duration (read how Discord handles millions of concurrent voice calls can be found in a [previous blog post](#)). This meant if one Icelandic user started a call with three users in Germany, the call could land on the Reykjavik server, and the German users would send their packets to Iceland and back. The "local" PoP was making things worse for everyone who wasn't local.

We had to revise the strategy; it turns out that a new PoP only helped if the calls hosted there stayed local, meaning for mixed-region calls, host placement mattered more than coverage. So, instead of filling coverage gaps, we shifted focus to replacing existing regions one at a time. Coverage in places like Reykjavik went onto the long-term roadmap, paired with smarter call placement logic.

Phase 1: Rotterdam

Phase 1 began in late April 2025, shifting Rotterdam traffic onto Cloudflare's Amsterdam PoP. Most ISPs in the region looked fine: ping was normal and quality metrics held.

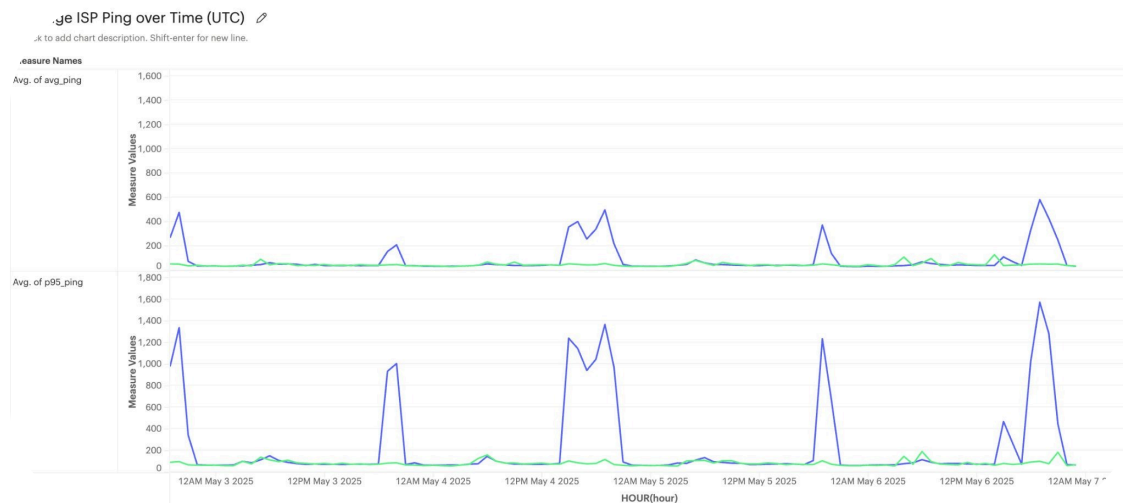


Discord

Open Discord

Within hours, users connecting via Orange, one of France's largest ISPs, were seeing call latency above one second during peak hours. Voice quality on Orange calls regressed 30% on freeze ratio, which was, unfortunately, in the Red.

We started digging to see what was up. Other ISPs in the region were healthy, other Cloudflare PoPs were healthy, so what was goin' on here? The problem turned out to be the Orange-to-Amsterdam path. No ISP runs a direct wire to every other network on the internet. Instead, traffic rides through a transit provider that bridges the gap in between. In this case, Orange's connectivity into Cloudflare's edge ran through Telia's transit backbone, and the Telia-to-Orange handoff was already saturated at peak. Every additional megabit we shifted onto Cloudflare made it worse.



About ten days in, we reverted. Rotterdam's Cloudflare instances were drained and we leaned on prior vendor capacity to absorb the Orange-bound traffic. Cloudflare started direct peering negotiations with Orange and recommended we deploy SFUs into their Paris and London PoPs to give Orange traffic shorter paths to a Cloudflare edge. Those PoPs came online over the following months, and everyone involved benefitted from these PoP changes!

This influenced our rollout process, too. Before Rotterdam, we'd been pacing rollouts by capacity readiness. After Rotterdam, we paced them by peering analysis: before any region got meaningful traffic, the team checked that Cloudflare's peering with that region's major ISPs had headroom. If it didn't, the region waited.

What Changed



Discord

[Open Discord](#)

Cloudflare's PoPs really do reach more places than hyperscale regions, and traffic from a closer PoP really does have lower latency. Because of this, we switched up how we thought about migration order. Iceland made it clear the closest PoP isn't always the best host for any given call, and Rotterdam showed that even when it *should* be, the network path can still be the problem.

The migration slowed down. The cadence we'd originally planned, large regions in months and full migration by year-end, gave way to a region-by-region rollout with peering checks up front. While we should have been doing it that way from the start, experimenting with other rollout methods was still beneficial in helping us determine the proper path forward.

We also knew by then that Cloudflare's operational mechanics weren't going to look like anything we'd run before. Most of the first months went into making basic deployment and host-lifecycle work there.

Discovery: Hosts Dial In

Discovery on our existing infrastructure was simple: provision a host and register it with our discovery service. Cloudflare inverts this; Cloudflare's scheduler brings machines up and reclaims them on its own timeline for maintenance or to shift traffic load, so hosts pop into existence with no pre-launch heads-up and the host has to introduce itself.

We built a discovery service to handle that. A voice host comes up on Cloudflare, calls in to register, re-registers periodically while running, and unregisters on shutdown. These registrations live in Valkey, a GCP memory store service, with a ten-minute TTL, so a host that stops checking in drops out automatically.

Our older service discovery had been built on etcd years ago for a much smaller fleet, and at 25,000 voice hosts on Cloudflare it was starting to strain. Discovery latencies were climbing and we were one large rollout away from breaking it. Storing Cloudflare hosts through Valkey kept the new growth off etcd, and over the course of the



migration we moved the legacy SFU records too, double-writing through the cutover before retiring etcd.

Discord

[Open Discord](#)

Hosts as Spot Instances

Cloudflare's deployment platform was optimized for short-lived V8 Isolates, and not originally intended for voice traffic. A Discord voice call runs on a single host that holds both a stateful control plane (who's in the call, what's muted, where streams flow) and a stateless media plane (the UDP packets for audio and video). The UDP transport stays open for the duration of the call, so recycling that host mid-deploy can drop the call.

Most clouds assume hosts stay put. A host gets a stable name on launch and we retire it on our own schedule. Cloudflare doesn't work that way, meaning their scheduler can load-shed a running host to reclaim it if the resource is needed elsewhere. On top of any reclaim events, our containers there get rebooted at least once a month as part of their regular maintenance.

Recreate, Not Restart

On Cloudflare, every code change is a rollout that recreates the container from the new image. We built a deployer that asks Cloudflare to recreate any host on the wrong (old) image and polls until each one comes back.

A coordinated recreate across a PoP could briefly drop the instance count to zero, forcing the container's supervisor to catch the shutdown signal and delay its actual exit by five minutes. Replacements come up during that window, the PoP stays populated, and call reconnects land on new hosts in the same region instead of getting bounced.

Two Workers, Then Four, Then Eight

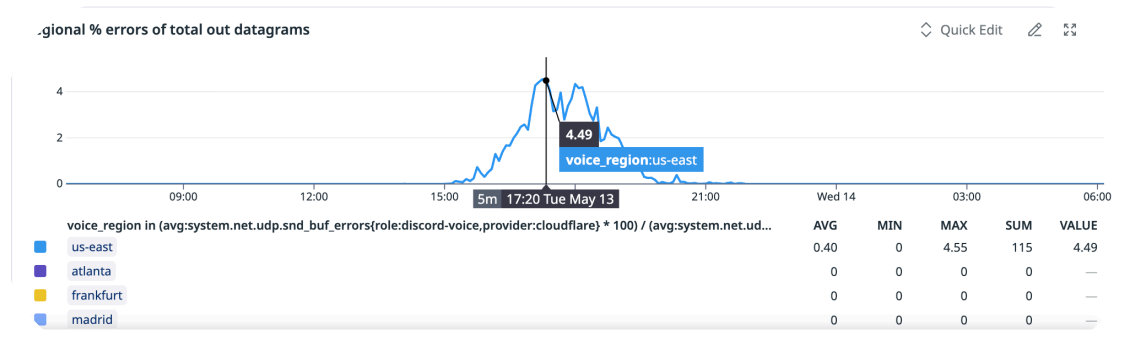
The first combined voice/worker container we shipped to Cloudflare ran four workers per host. That was the average density that worked everywhere else... but it didn't work here.



Discord

[Open Discord](#)

In early May 2025, US East was seeing consistent packet loss of 1.5–2%, against a baseline of less than 0.5% on the prior provider across the Cloudflare path. We managed to trace it back to queue contention. On bare metal, each worker had its own NIC transmit queue, but Cloudflare's container runtime didn't yet expose multi-queue NICs to our containers, so all four workers were sharing a single queue. The kernel started dropping UDP packets at the send-buffer layer during peak hours.



As a short-term fix, we halved the density. We re-released with two workers per host, and packet loss dropped back to parity with the prior provider. While the per-host availability got worse, the path was viable again.

Cloudflare worked on the underlying buffer issue over the following weeks. By early June 2025 they had a UDP buffer fix, and we re-enabled four workers in Amsterdam as a pilot. By late September, an eight-worker configuration was the global default.

We expected to start dense and stay dense. Instead, we shipped at four, shrank to two for a few weeks of debugging, and rose back up to eight.

By early 2026, the migration mechanics were largely behind us. Two issues remained; both took us deeper into Cloudflare's infrastructure than the migration itself ever had.

The Twenty-five-second(!) Hangs



Discord

[Open Discord](#)

The first issue showed up as outages that weren't *really* outages.

Voice instances in some PoPs started becoming unresponsive for

upwards of twenty-five seconds at a time, then they left no the logs about this hang. Cloudflare's Los Angeles PoP saw it more often than others.

Cloudflare had been getting similar signals from another customer, and their kernel team had independently picked up VM-level Input/Output stalls on the host. The issue was page cache buildup on newer disks. Our write-heavy workload would fill the cache, then the kernel would eventually flush several gigabytes at once, and every other writer on the block device stalled until the flush completed. PoPs with newer hardware ran more of our workload and stalled more often.

Once identified, Cloudflare was able to fix it on their side. First, with a tuning change to flush dirty pages more frequently, then with a more complicated change to their VMM's sync Input/Output path. Our contribution was reporting the symptom with enough telemetry for Cloudflare's team to correlate it against their host metrics, bringing the fix to all of Cloudflare's user base.

The European Ping Spikes

When we migrated Bucharest and Stockholm onto the new Rust-based SFU on Cloudflare in late 2025, both regions' metrics started slipping. Ping climbed during peak hours, voice quality degraded over the following weeks, and by February, client-side ping latencies were spiking above half a second. Reports about ping spikes and robotic audio even came in through our support channels and Reddit.

We shifted some European traffic off Cloudflare while we investigated. It took several weeks and turned into a longer joint debug than we'd done with Cloudflare before. The early theories were the usual suspects: a routing anomaly, a peering issue, a DDoS hitting our ports. None of them held up.

The Probe

Our investigation breakthrough came from a passive probe Cloudflare's own performance team wrote.



Discord

Open Discord

Our voice health-checks carry a fixed payload signature identifiable on the wire, without inspecting connection state. Cloudflare built an eBPF program that hooked into their host's network stack and timestamped every Discord health-check passing through. Comparing arrival and departure timestamps at the NIC gave them on-metal processing time for every health-check on every host we ran, without touching our traffic.

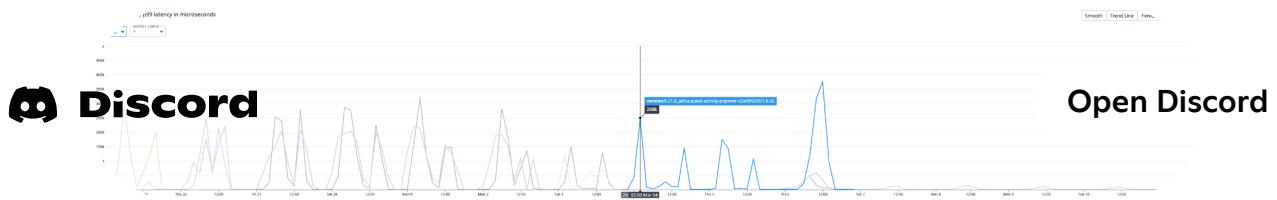
The probe segmented further and found that the physical NIC to virtual NIC was clean, virtual NIC back to physical was clean. The time was being spent inside the VM, meaning it was inside our own application.

We gave Cloudflare's team direct access to a container in one of the affected regions. Within minutes, they observed futex stalls of up to 860 milliseconds inside our process. Per-thread CPU sampling over the following days showed one thread pegged at 100% on a single core during every spike window. The socket receive buffers grew during those same windows. A couple of days later, we had flame graphs showing the failure mode end to end.

Write Starvation in Our App

The first root cause was in our code. Our SFU runs eight worker threads, each with its own UDP socket, single-threaded Tokio runtime, and select! event loop. The two things that matter here are a recv future, which pulls packets off the socket, and a flush timer, which pushes outgoing packets back out. When the receive buffer is non-empty, the recv future is always ready, and a future that's always ready gets polled repeatedly. The event loop ends up stuck polling recv, the flush timer never fires, and starves the outgoing sends.

Tokio has a 128-poll yield budget meant to prevent this, but during peak traffic, 128 polls happen in *microseconds*, and the recv future is still ready when the budget runs out. The thread burned 100% CPU polling recv. Our media_batch.latency_microseconds metric, or how long a packet sits in the send queue before flush, climbed to hundreds of milliseconds during spike windows against a baseline of about one millisecond.



The fix was a nine-millisecond budget. If it had been nine milliseconds since the last `send()` flush, the `recv` of the `select!` was disabled for that poll, forcing the flush timer to run. We shipped that in early March. While the p99 latency dropped right away, the spikes still showed up at peak hours.

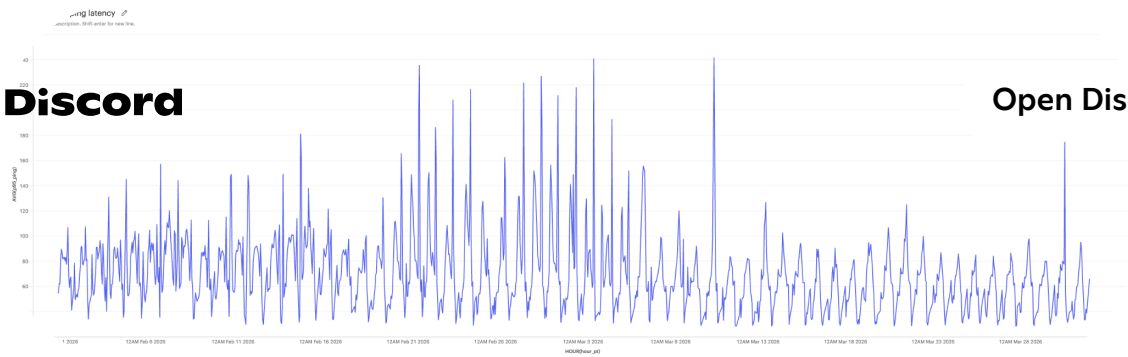
Softirq noisy neighbor

The remaining jitter came from Cloudflare's infrastructure. At the time, their VMs exposed a single virtio-net queue per VM, so all Receive softirq processing for a VM landed on a single vCPU. When our worker threads were scheduled onto that same vCPU, softirq processing preempted them mid-execution and stalled them for the duration of the burst. Flame graphs taken after our `recv`, and `send` fix showed both `net_rx_action` and `local_bh_enable` frames lining up exactly with the residual spikes.

Cloudflare identified the issue. To address it, we deployed a CPU affinity change in our container entrypoint by using `taskset` to pin our worker threads off the vCPU Cloudflare had pinned the Receive IRQ to, and leveraged Receive Packet Steering to spread softirq processing across other cores while keeping workers off them. The first peak after both fixes shipped showed no latency spikes. We moved European traffic back onto Cloudflare and retired the fallback capacity.



Discord

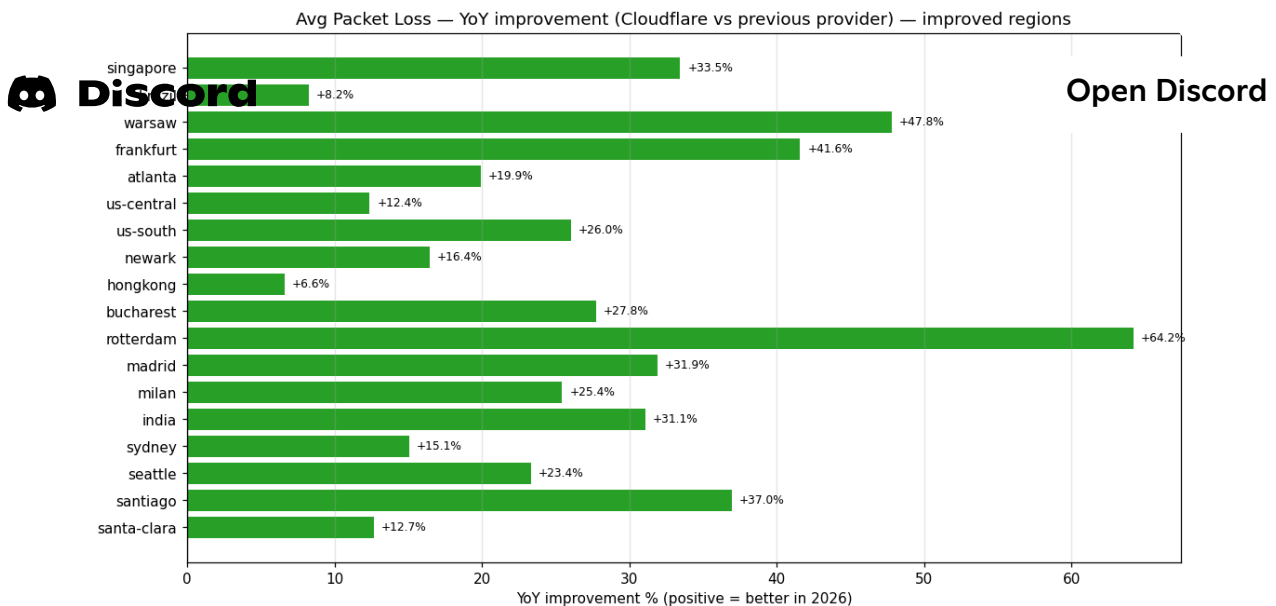


Open Discord



Running real-time media on shared hardware exposes scheduling and queue-contention behaviors that don't matter for HTTP or static workloads, and the European spikes were the first time we had to dig into that on Cloudflare. They're currently migrating their container platform onto a hypervisor that supports multi-queue virtio-net, which should allow us to retire the CPU affinity workaround in the near future.

A year in, more than 80% of Discord's voice and video traffic now runs on Cloudflare. Against our previous vendor, more than 70% of the migrated regions show net quality improvements year-over-year. The strongest gains are in Europe and Latin America, where packet loss is down 20-60% across the multiple regions, and in Santiago, we're seeing expand ratio (the measurement of audio frames we stretch to cover gaps) is down 40%.



A few of the regions still remain in focus in the next quarter. The regions that motivated this blog post in the first place, like Reykjavik and Auckland, are still ahead of us, alongside the call placement work that Iceland made the case for.

Moving forward, our plan of attack will be slow ramps with metrics in the loop, peering analysis before any region goes live, and joint debugging with Cloudflare when something surprises us. Cloudflare will be spinning up new PoPs; our addressable footprint grows with theirs.

Our goal hasn't changed: Closer servers make better calls, and with Cloudflare providing us with more places to put them, we'll be able to help make better game night VC memories for millions of players across the world.

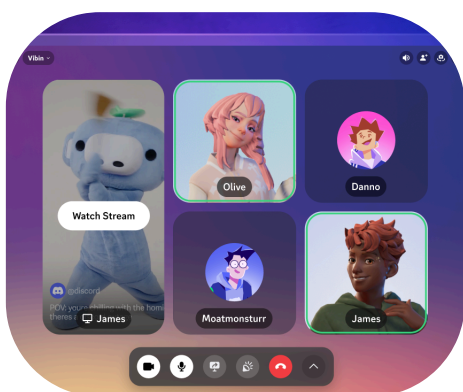
If this kind of work sounds interesting, take a look at [our careers page](#).



David Chen

Engineering Manager, Audio/Video Infrastructure

RELATED ARTICLES



Product &
Features

**Discord is
Now on
Meta...**



Product &
Features

**Discord
Update:
June 25,...**





Discord

Product &
Features

Introducing:
You Bar

Open Discord

Language	Product	Company	Resources	Policies
	Download	About	Support	Terms
	Nitro	Jobs	Safety	Privacy
	Status	Brand	Blog	Cookie Settings
	App	Newsroom	Creators	Guidelines
Social	Directory		Community	Acknowledgement
			Developers	Licenses
			Quests	Company Information
			Official 3rd Party Merch	
			Feedback	

Discord